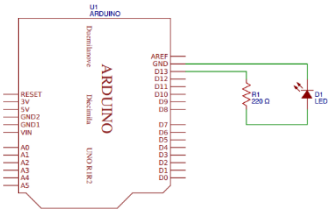


Part1_TP N°3a

Gestion des interruptions internes
Application web (IHM)
Durée max conseillée : 1h



Partie lecture durée conseillée : 15 min

Objectif general:

Ce TP prolonge les TP1 et TP2. Il aborde la gestion des interruptions internes du microcontrôleur — limitée ici au Timer2 — afin de générer une base de temps précise sans bloquer le programme principal.

1 Préambule

1.1 Objectifs pédagogiques

- Comprendre le principe d'une interruption interne (timer) et la différence avec une temporisation par delay().
- Identifier les registres de contrôle du Timer2 (TCCR2A, TCCR2B, TCNT2, OCR2A, TIMSK2, TIFR2) et leur rôle.
- Configurer le Timer2 en mode CTC (Clear Timer on Compare Match) pour générer une interruption périodique précise.
- Écrire une routine de service d'interruption (ISR) et échanger une information avec le programme principal via une variable volatile.

1.2 Prérequis

Les connaissances développées dans les TP1 et TP2 sont considérées comme acquises.

- Utiliser l'IDE Arduino : créer, compiler et téléverser un programme (TP1).
- Configurer une entrée/sortie numérique et utiliser pinMode(), digitalRead(), digitalWrite() (TP1).
- Utiliser une interruption externe : attachInterrupt(), digitalPinToInterrupt(), une ISR, une variable volatile (TP2).
- Utiliser le Moniteur Série (TP1) et structurer une trame JSON (TP2).
- Comprendre les principes élémentaires du C/C++ : variables, fonctions, conditions, boucles.

1.3 Introduction

Dans le TP2, la réactivité du système reposait sur des interruptions externes déclenchées par un événement matériel (bouton, encodeur). Ce TP introduit les interruptions internes, générées par un timer du microcontrôleur lui-même, indépendamment de toute broche d'entrée.

1.3.1 Problématique

Comment générer une tâche périodique précise sans bloquer l'exécution du programme principal ?

2 Matériel nécessaire

2.1 Carte de développement et poste de travail

La plateforme utilisée est la même que dans les TP1 et TP2 : Arduino Uno (ATmega328P).

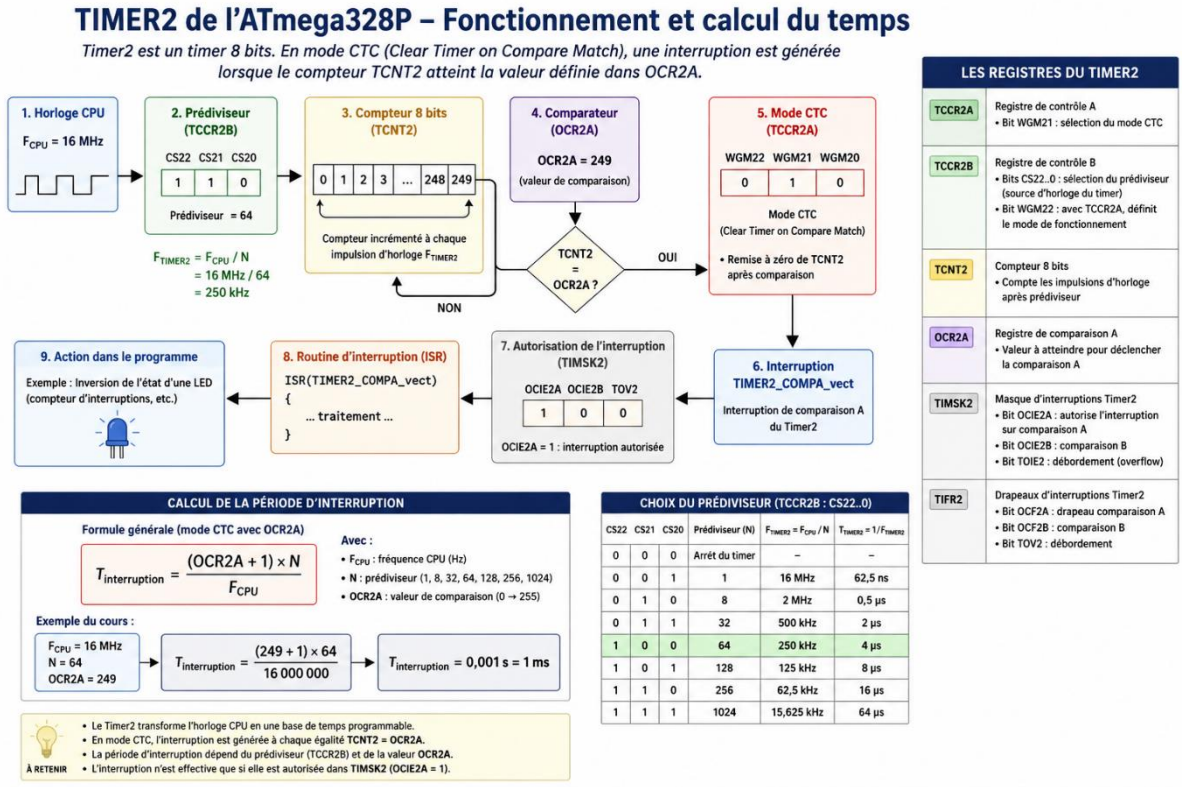
Élément	Rôle
Arduino Uno	Microcontrôleur et plateforme de développement
Câble USB	Alimentation, programmation et liaison série avec le PC
Ordinateur (Windows)	IDE Arduino
1 LEDs (LED_BUILTIN)	LED 13 : tout-ou-rien

2.2 Broches utilisées

Fonction	Broche proposée	Remarque
LED de test Timer2	D4	base de temps = 1s

3 Générer une base de temps avec le Timer2

3.1 Principe de l'interruption



3.1.1 Fonctionnement du Timer2 et génération d'une base de temps

Le **Timer2 de l'ATmega328P** est un compteur matériel de **8 bits** permettant de mesurer ou de générer des intervalles de temps sans que le programme principal ait besoin d'utiliser une temporisation bloquante comme delay().

Le fonctionnement du Timer2 repose sur plusieurs registres qui interviennent successivement dans la génération de la base de temps.

3.1.1.1 L'horloge du microcontrôleur

Le Timer2 utilise l'horloge du microcontrôleur, $F_{CPU} = 16\,MHz$ généralement de **16 MHz** sur une carte Arduino Uno. Cette horloge constitue le point de départ du calcul temporel.

Cependant, le compteur n'est pas nécessairement incrémenté à chaque période de l'horloge CPU. Un **prédiviseur** permet de réduire la fréquence appliquée au Timer2.

3.1.1.2 Le prédiviseur — registre TCCR2B

Les bits CS22, CS21 et CS20 du registre TCCR2B permettent de sélectionner le prédiviseur.

Dans notre exemple :

TCCR2B |= (1 << CS22);

TCCR2B |= (1 << CS21);

correspond à un prédiviseur de **64**.

La fréquence appliquée au compteur est donc : $F_{Timer} = \frac{16\,MHz}{64} = 250\,kHz$

Avec une horloge CPU de 16 MHz :

Chaque incrément du compteur correspond donc à : $T_{tick} = \frac{1}{250\,000} = 4\,\mu s$

Le prédiviseur permet ainsi d'adapter la vitesse de comptage du Timer2 à l'intervalle de temps que l'on souhaite obtenir.

3.1.1.3 Le compteur TCNT2

Le registre TCNT2 contient la valeur courante du compteur. Comme le Timer2 est un compteur **8 bits**, sa valeur peut aller de **0 à 255**.
En mode CTC, le compteur est incrémenté à chaque impulsion de l'horloge du Timer2 :
0 → 1 → 2 → 3 → … → 248 → 249
Le registre TCNT2 représente donc directement le nombre d'impulsions d'horloge comptées depuis la dernière remise à zéro.

3.1.1.4 La valeur de comparaison — registre OCR2A

Le registre OCR2A contient la valeur à laquelle le compteur doit effectuer une comparaison.

$$N = \frac{1\,ms}{4\,\mu s} = 250$$

Dans notre exemple :
OCR2A = 249;
Le matériel compare en permanence si TCNT2 = OCR2A.Lorsque : TCNT2 = OCR2A une **comparaison A** est détectée.
Il faut tenir compte du fait que le compteur part de 0. Avec OCR2A = 249, il faut donc **250 impulsions** pour atteindre cette valeur :249+1=250

La durée entre deux comparaisons est alors :
T_interruption = 250 × 4 µs
T_interruption = 1000 µs = 1 ms

3.1.1.5 Le mode CTC — registre TCCR2A

Le registre TCCR2A permet notamment de sélectionner le mode de fonctionnement du Timer2.
Le bit WGM21 est positionné à 1 afin de sélectionner le **mode CTC (Clear Timer on Compare Match)**.
Dans ce mode, lorsque TCNT2 atteint la valeur de OCR2A, le compteur est automatiquement remis à zéro et recommence un nouveau cycle.
Le Timer2 produit ainsi une succession régulière de périodes de 1 ms dans notre exemple :
Le mode CTC est particulièrement intéressant pour créer une **base de temps périodique**.

3.1.1.6 L'autorisation de l'interruption — registre TIMSK2

La comparaison entre TCNT2 et OCR2A ne suffit pas à déclencher automatiquement l'exécution d'une routine d'interruption.
L'interruption de comparaison A doit être autorisée dans le registre TIMSK2.
Le bit OCIE2A doit être positionné à 1 :
TIMSK2 |= (1 << OCIE2A);
Le Timer2 peut alors demander au processeur d'exécuter la routine d'interruption associée à la comparaison A.

3.1.1.7 La routine d'interruption

L'interruption de comparaison A du Timer2 est associée à la routine :

```
ISR(TIMER2_COMPA_vect)
{
```

```
    // traitement
}
```

Cette fonction est exécutée automatiquement à chaque comparaison entre TCNT2 et OCR2A.

Dans notre exemple, elle est donc appelée **toutes les 1 ms**.

On peut par exemple utiliser cette interruption pour incrémenter un compteur logiciel :

```
ISR(TIMER2_COMPA_vect)
{
    compteur++;
}
```

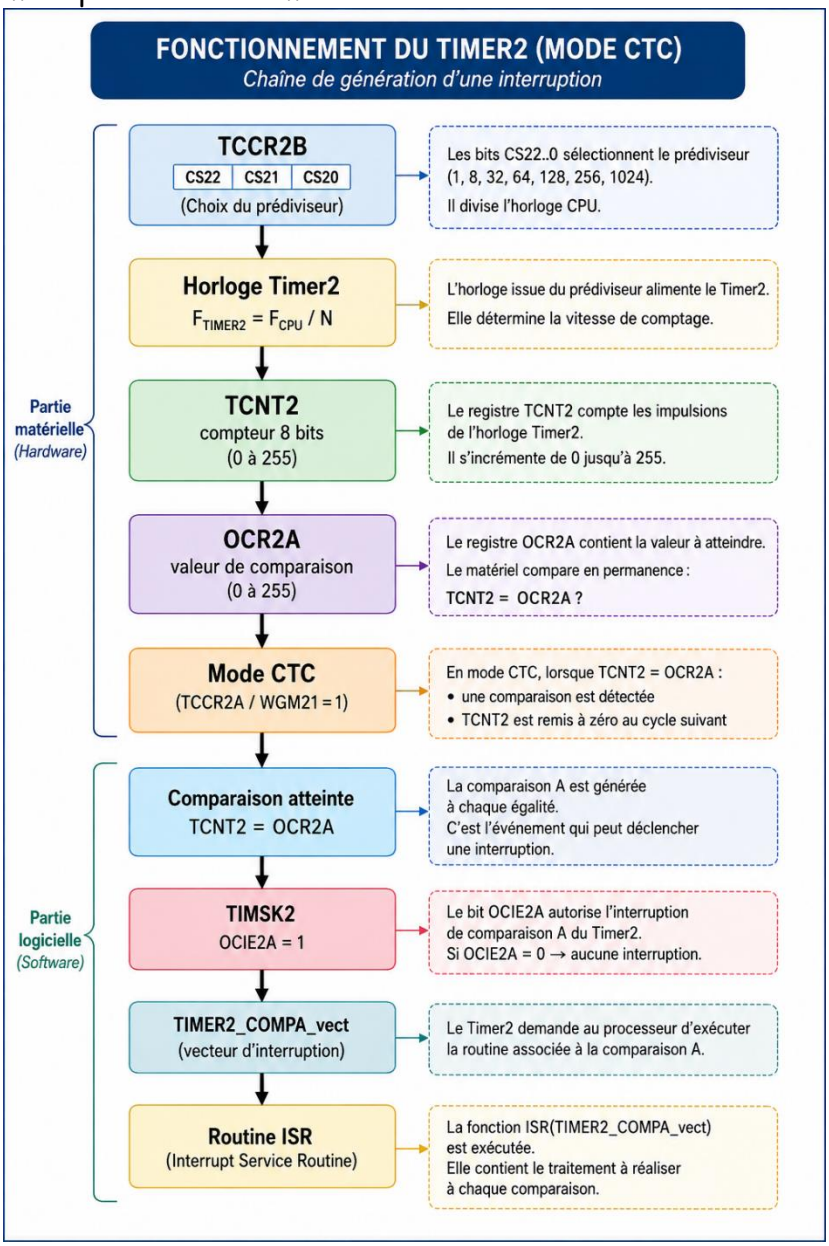
Le Timer2 fournit alors une référence temporelle au programme.

Après 1000 interruptions : 1000x1 ms=1s

Le programme peut donc utiliser cette base de temps pour réaliser une action toutes les secondes, comme inverser l'état d'une LED.

3.1.1.8 Relation entre les différents registres

Le fonctionnement peut être résumé ainsi :



3.1.1.9 *À retenir*

Le **Timer2 transforme l'horloge du microcontrôleur en une base de temps programmable**.

- TCCR2B → définit notamment le **prédiviseur** ;
- TCNT2 → contient la **valeur courante du compteur** ;
- OCR2A → définit la **valeur de comparaison** ;
- TCCR2A → permet notamment de sélectionner le **mode CTC** ;
- TIMSK2 → permet d'**autoriser l'interruption** de comparaison ;
- ISR(TIMER2_COMPA_vect) → contient le **traitement exécuté périodiquement**.

Pour un fonctionnement avec F_CPU = 16 MHz, un prédiviseur de 64 et OCR2A = 249 : T_interruption=1 ms
Le Timer2 génère ainsi **1000 interruptions par seconde**, fournissant au programme une base de temps précise et régulière.

3.1.1.10 *En résumé.*

Un timer est un registre interne (périphérique) au micro-contrôleur ATMega328.
C'est en fait un compteur qui s'incrémente (ou se décrémente) à chaque impulsion d'un signal d'horloge (tick), interne ou externe.
Ainsi la mesure du temps qui s 'écoule est possible. D'où le nom donné à ce type de périphérique Timer/Counter.
Le temps que met ce Timer/Counter pour compter (ou décompter) dépend de la fréquence (ou de la période) des ticks. Le comportement du Timer/Counter est directement lié à la programmation de registres de contrôle, accessibles comme d'habitude en lecture/écriture.

Si le registre timer est de taille 8 bits, il est capable de compter de 0 à 255. Lorsque qu'il arrive à 255 il repasse à 0 (2n=0). On dit qu'il a débordé(overflow). Un flag particulier d'un registre de contrôle propre au timer détecte ce débordement en passant de 0 à 1.
Afin de « ralentir » la durée de comptage le Timer/Counter dispose de pré-diviseurs programmables (prescalers).

3.1.2 Intérêt pratique.

Un timer utilisé par millis(), compte sans cesse en tâche de fond, ce qui ne bloque pas le programme en cours d'exécution. Ce qui n'est pas le cas de la fonction delay() qui est bloquante.

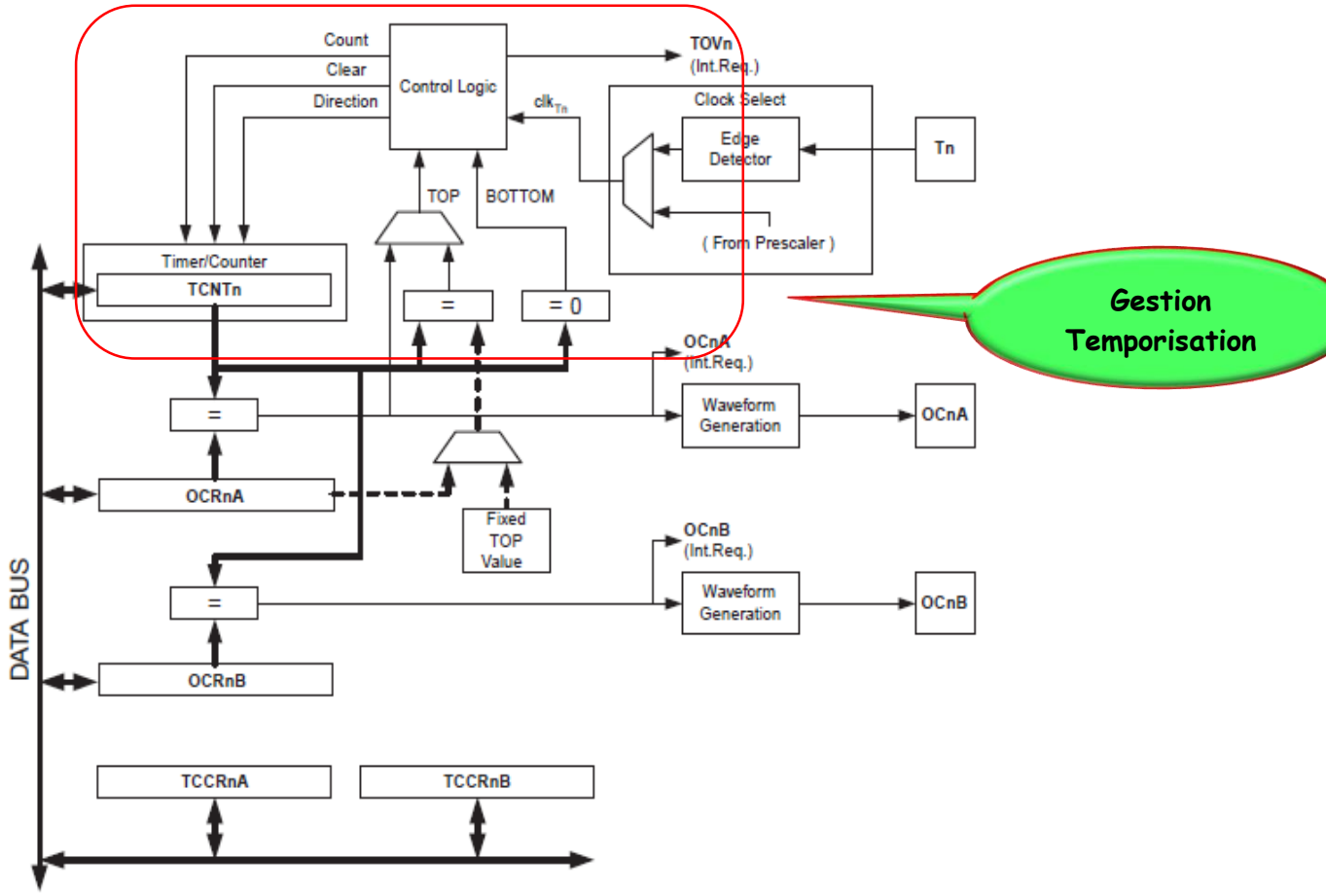
3.1.3 Les 3 modules Timers de l'ATMega328

Le Timer0 de taille 8bits. Utilisé entre autres par les fonctions delay(), millis() et micros().
Le Timer1 de taille 16 bits. Utilisé pour produire des signaux de commande de servomoteurs…(PWM sur pins 9 et 10 de la Uno)
Le Timer2 de taille 8bits (PWM sur pins 3 et 11 de la Uno).

3.1.4 Timer/Counter2 (comptage 8 bits)

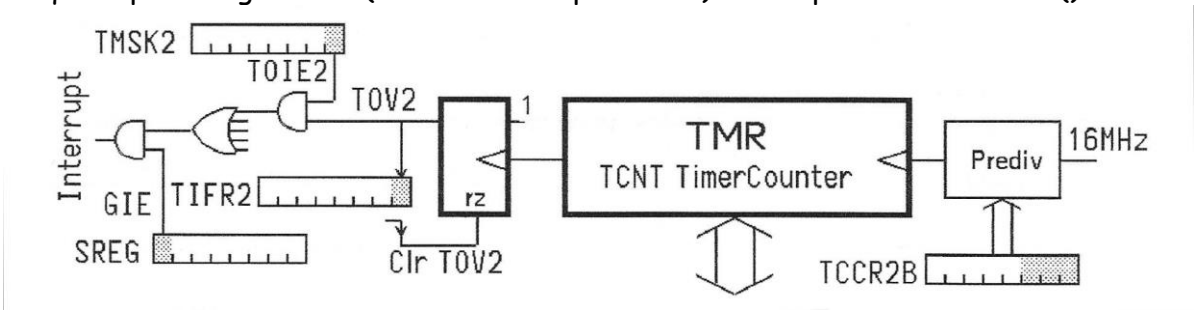
C'est un module Timer/Counter avec registre de comptage 8 bits.

La structure générale du module Timer/Counter2 est donnée dans le schéma ci-dessous.
Le registre de comptage se nomme **TCNT2** (registre 8 bits).



3.1.5 Architecture du contrôle du timer2 (identique pour Timer0 et Timer1).

Toutes les demandes d'interruption arrivent sur une porte OU et sont éventuellement bloquées par le signal ***GIE*** (Global Interrupt Enable) activé par la fonction sei().



3.1.6 Les registres de contrôle.

Nous nous intéresserons au seul timer2. (Timer0 mêmes registres…)

TCNT2 – Timer/Counter Register

Bit	7	6	5	4	3	2	1	0	
(0xB2)	TCNT2[7:0]								TCNT2
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	

TCCR2A – Timer/Counter Control Register A

Bit	7	6	5	4	3	2	1	0	
(0xB0)	COM2A1	COM2A0	COM2B1	COM2B0	–	–	WGM21	WGM20	TCCR2A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	

TCCR2B – Timer/Counter Control Register B

Mode normal de fonctionnement en « temporisateur » (timer).

- Le registre **TCNT2** est incrémenté de 1 à chaque événement de comptage.
- Le registre ne revient à 0 qu'après un débordement (passage de 0xFF à 0x00).

→ Configuration de ce mode via les registres TCCR2A et TCCR2B

Mode	WGM2	WGM1	WGM0	Timer/Counter Mode of Operation	TOP	Update of OCRx at	TOV Flag Set on(1)(2)
0	0	0	0	Normal	0xFF	Immediate	MAX

Prescaler

En temporisateur, le registre de comptage **TCNT2** est incrémenté en fonction des cycles horloge.

L'incrément peut être à chaque cycle horloge (« tick ») (pas de prescaler) ou bien à une fréquence moindre (prescaler nécessaire).

*Rappelons que le cycle horloge est de **1/16 µseconde**. On peut donc mettre un diviseur de fréquence entre l'horloge (16Mhz) et l'incrément du registre de comptage.*

Table 17-9. Clock Select Bit Description

CS22	CS21	CS20	Description
0	0	0	No clock source (Timer/Counter stopped).
0	0	1	clk_{T2S}/(No prescaling)
0	1	0	clk_{T2S}/8 (From prescaler)
0	1	1	clk_{T2S}/32 (From prescaler)
1	0	0	clk_{T2S}/64 (From prescaler)
1	0	1	clk_{T2S}/128 (From prescaler)
1	1	0	clk_{T2S}/256 (From prescaler)
1	1	1	clk_{T2S}/1024 (From prescaler)

Partie activités durée conseillée : 45 min

Vous allez réaliser un programme permettant de faire fonctionner une LED à intervalle régulier tout en laissant le microcontrôleur disponible pour d'autres tâches.

4 Observer les limites de delay()

Programmer une LED pour qu'elle change d'état toutes les 500 ms :

```
void loop()
{
    digitalWrite(4, HIGH);
    delay(500);

    digitalWrite(4, LOW);
    delay(500);
}
```

Puis ajouter dans loop() une seconde tâche :

```
Serial.println("Programme principal");
```

Questions

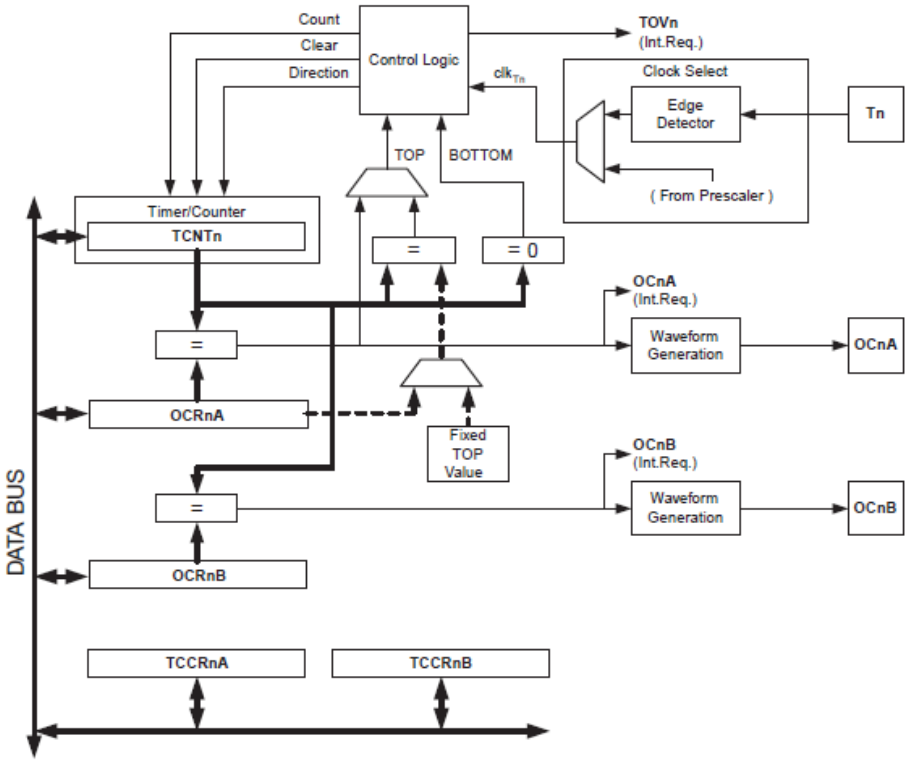
1- Que se passe-t-il pendant delay() ?
2- Le microcontrôleur peut-il exécuter simultanément une autre tâche ?
3- Pourquoi cette solution est-elle problématique dans un système embarqué comportant plusieurs tâches ?

4.1 Qu'est-ce qu'un timer ?

Un timer est un registre interne (périphérique) au microcontrôleur ATmega328P. C'est un compteur qui s'incrémente à chaque impulsion d'un signal d'horloge (tick), ce qui permet de mesurer le temps qui s'écoule — d'où le nom Timer/Counter.
Si le registre est codé sur 8 bits, il compte de 0 à 255 puis revient à 0 : on dit qu'il a débordé (overflow). Un flag dédié détecte ce débordement. Pour "ralentir" la durée de comptage, chaque Timer/Counter dispose de pré-diviseurs programmables (prescalers). Un timer utilisé par millis() compte en tâche de fond sans bloquer le programme — contrairement à delay(), qui est bloquante.

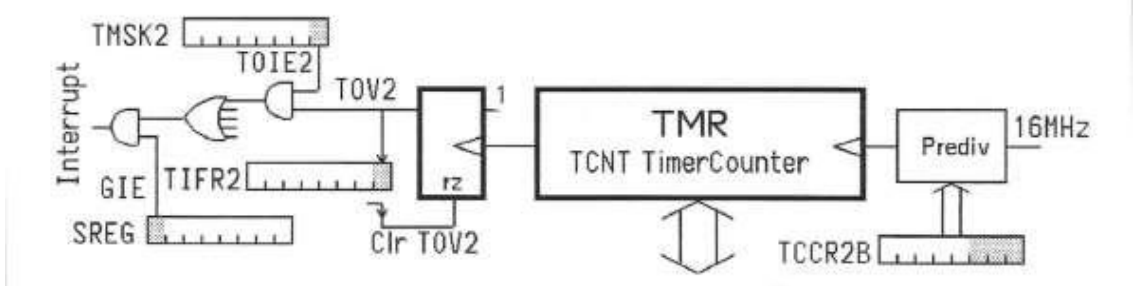
Timer	Taille	Usage typique sur Uno
Timer0	8 bits	Utilisé par delay(), millis(), micros()
Timer1	16 bits	PWM sur pins 9 et 10 ; commande de servomoteurs

Timer2	8 bits	PWM sur pins 3 et 11 — utilisé dans ce TP en mode interruption



Structure générale d'un module Timer/Counter (générique, ici Timer2)

Toutes les demandes d'interruption arrivent sur une porte OU et sont éventuellement bloquées par le signal GIE (Global Interrupt Enable), activé par la fonction sei().



Architecture du contrôle des interruptions du Timer2 (identique pour Timer0 et Timer1)

4.2 Le mode CTC (Clear Timer on Compare Match)

En mode CTC, le compteur TCNT2 est incrémenté à chaque tick jusqu'à atteindre la valeur définie dans le registre de comparaison OCR2A. Lorsque TCNT2 == OCR2A, une interruption est générée et le compteur repart de 0 — sans qu'aucune instruction du programme principal n'ait besoin d'intervenir.

Mode	WGM2	WGM1	WGM0	Timer/Counter Mode of Operation	TOP	Update of OCRx at	TOV Flag Set on[1][2]
0	0	0	0	Normal	0xFF	Immediate	MAX

Registre TCCR2A — sélection du mode Timer/Counter (WGM2:0)

Table 17-9. Clock Select Bit Description

CS22	CS21	CS20	Description
0	0	0	No clock source (Timer/Counter stopped).
0	0	1	clk_{T2S}/(No prescaling)
0	1	0	clk_{T2S}/8 (From prescaler)
0	1	1	clk_{T2S}/32 (From prescaler)
1	0	0	clk_{T2S}/64 (From prescaler)
1	0	1	clk_{T2S}/128 (From prescaler)
1	1	0	clk_{T2S}/256 (From prescaler)
1	1	1	clk_{T2S}/1024 (From prescaler)

Registre TCCR2B — sélection du prédiviseur (CS22:0)

Le détail bit-à-bit des registres TCNT2, TCCR2A, TCCR2B, TIFR2 et TIMSK2 est fourni en Annexe A.

4.3 Exercice de dimensionnement

Le quartz de l'ATmega328P a une fréquence FQ = 16 MHz.

Exercice
1. Calculer le temps tOV1 et tOV2 mis par les Timer1 (16 bits) et Timer2 (8 bits) respectivement pour déborder, sans prédiviseur.
2. Avec un prédiviseur de 64 sur le Timer2, déterminer la valeur à placer dans OCR2A pour obtenir une interruption toutes les 1 ms.

5 Travail pratique n°1 — Base de temps avec Timer2

Objectif : utiliser le Timer2 de l'ATmega328P pour générer périodiquement une interruption et réaliser une fonction temporelle sans bloquer le programme principal.

Mise en situation : on souhaite faire clignoter une LED à intervalles réguliers tout en permettant au programme principal de poursuivre d'autres traitements. Le clignotement ne devra pas être réalisé avec delay() ; il sera déclenché par une interruption interne générée par le Timer2.

Configuration retenue

Registre	Configuration	Rôle

TCCR2A	WGM21 = 1	Mode CTC
TCCR2B	CS22 = 1, CS21 = 1	Prédiviseur ÷64
OCR2A	249	Valeur de comparaison (1 ms)
TIMSK2	OCIE2A = 1	Autorisation de l'interruption
TCNT2	0	Compteur initial

Programme à compléter

```
#include <avr/interrupt.h>

// Variable partagée entre le programme principal et l'ISR.
// volatile indique au compilateur qu'elle peut être modifiée
// à tout moment par une interruption.
volatile unsigned long compteur = 0;

// ROUTINE D'INTERRUPTION DU TIMER2
// Exécutée automatiquement à chaque comparaison TCNT2 == OCR2A.
// Avec la configuration ci-dessus : 1 interruption = 1 ms.
ISR(TIMER2_COMPA_vect)
{
    compteur++;
}

void setup()
{
    pinMode(4, OUTPUT);
    setupTimer2();
    Serial.begin(9600);
}

void loop()
{
    // Le programme principal continue de fonctionner
    // pendant que le Timer2 génère ses interruptions.
    Serial.println("Programme principal");

    // 1000 interruptions de 1 ms = 1000 ms = 1 seconde.
    if (compteur >= /* à compléter */)
    {
        compteur = 0;
        digitalWrite(4, !digitalRead(4));
    }
```

```
}

void setupTimer2()
{
    cli();                          // désactivation temporaire des
interruptions

    TCCR2A = 0;
    TCCR2B = 0;
    TCNT2  = 0;                     // remise à zéro du compteur

    OCR2A = /* à compléter */;      // valeur de comparaison (voir 3.4)

    TCCR2A |= (1 << WGM21);         // mode CTC
    TCCR2B |= (1 << CS22);          // prédiviseur 64
    TCCR2B |= (1 << CS21);

    TIMSK2 |= (1 << OCIE2A);        // autorise l'interruption de comparaison
A

    sei();                          // réactivation des interruptions
}
```

Travail demandé

1. Identifier le Timer2 : préciser qu'il s'agit d'un timer 8 bits, et identifier le rôle de chacun des registres TCCR2A, TCCR2B, TCNT2, OCR2A, TIMSK2, TIFR2.

2. Configurer Timer2 en mode CTC : le compteur TCNT2 est incrémenté jusqu'à atteindre la valeur définie dans OCR2A, ce qui génère une interruption.

3. Compléter les deux zones à compléter du programme (OCR2A, condition sur compteur) à partir de l'exercice de dimensionnement (3.4).

4. Téléverser le programme et vérifier que la LED clignote toutes les secondes tout en observant "Programme principal" défiler dans le Moniteur Série.

5. Décrire le fonctionnement observé.

Questionnaire

1. Quel est le rôle du registre OCR2A ?

2. Pourquoi la variable compteur est-elle déclarée volatile ?

3. Que se passerait-il si Serial.println() était appelé à l'intérieur de l'ISR plutôt que dans loop() ?

4. Quel est l'intérêt du mode CTC par rapport au mode normal (comptage jusqu'à débordement 0xFF) ?

5. Pourquoi faut-il appeler sei() à la fin de setupTimer2() ?

6. En quoi cette approche est-elle différente de l'interruption externe vue en TP2 (attachInterrupt sur un bouton) ?

6. Conclusion du TP

Ce TP a complété la panoplie de mécanismes de réactivité d'un système embarqué : après la scrutation (TP1) et les interruptions externes (TP2), vous avez mis en œuvre une interruption interne générée par un timer, capable de produire une base de temps précise sans jamais bloquer le programme principal.

La démarche acquise — configurer un périphérique interne via ses registres de contrôle, isoler le traitement d'une interruption dans une ISR minimale.

7 Annexe A — Détail des registres du Timer2

Représentation bit-à-bit des registres utilisés (issue de la documentation ATmega328P).

TCNT2 – Timer/Counter Register

Bit	7	6	5	4	3	2	1	0	
(0xB2)	TCNT2[7:0]								TCNT2
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	

TCCR2B – Timer/Counter Control Register B

Bit	7	6	5	4	3	2	1	0	
(0xB1)	FOC2A	FOC2B	–	–	WGM22	CS22	CS21	CS20	TCCR2B
Read/Write	W	W	R	R	R	R	R/W	R/W	

TIMSK2 – Timer/Counter2 Interrupt Mask Register

Bit	7	6	5	4	3	2	1	0	
(0x70)	–	–	–	–	–	OCIE2B	OCIE2A	TOIE2	TIMSK2
Read/Write	R	R	R	R	R	R/W	R/W	R/W	

TIFR2 – Timer/Counter2 Interrupt Flag Register

Bit	7	6	5	4	3	2	1	0	
0x17 (0x37)	–	–	–	–	–	OCF2B	OCF2A	TOV2	TIFR2
Read/Write	R	R	R	R	R	R/W	R/W	R/W	

TCCR2A – Timer/Counter Control Register A

Bit	7	6	5	4	3	2	1	0	
(0xB0)	COM2A1	COM2A0	COM2B1	COM2B0	–	–	WGM21	WGM20	TCCR2A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	

Tableau de synthèse — Interruptions internes Arduino
Les interruptions internes sont généralement liées à des **événements générés à l'intérieur du microcontrôleur**, notamment par les temporisateurs.

API / fonction	Rôle	Syntaxe générique	Exemple
`ISR()`	Définir la fonction exécutée lors d'une interruption	`ISR(vecteur) { ... }`	`ISR(TIMER1_COMPA_vect) { compteur++; }`
`cli()`	Désactiver globalement les interruptions	`cli();`	`cli();`
`sei()`	Réactiver globalement les interruptions	`sei();`	`sei();`
`noInterrupts()`	Désactiver les interruptions	`noInterrupts();`	`noInterrupts();`

API / fonction	Rôle	Syntaxe générique	Exemple
`interrupts()`	Réactiver les interruptions	`interrupts();`	`interrupts();`
`volatile`	Indiquer qu'une variable peut être modifiée par une interruption	`volatile type variable;`	`volatile int compteur = 0;`

Exemple : interruption interne par temporisateur — Arduino Uno

```
// Variable globale modifiée par une interruption.
// Le mot-clé volatile indique au compilateur que cette variable
// peut changer à tout moment, indépendamment du programme principal.
volatile int compteur = 0;


void setup() {

  // Initialisation de la liaison série à 115200 bauds.
  Serial.begin(115200);


  // Désactive temporairement toutes les interruptions.
  // On évite ainsi qu'une interruption survienne pendant
  // la configuration du Timer1.
  noInterrupts();


  // ============================================================
  // CONFIGURATION DU TIMER1
  // ============================================================

  // Registre de configuration A du Timer1.
  // On met tous les bits à 0 pour repartir d'une configuration connue.
  TCCR1A = 0;

  // Registre de configuration B du Timer1.
  // Même principe : on remet le registre à zéro.
  TCCR1B = 0;


  // Valeur de comparaison du Timer1.
  // Lorsque le compteur du Timer1 atteint cette valeur,
  // une interruption sera générée.
  OCR1A = 15624;
```

```
    // ---------------------------------------------------------
    // Sélection du mode CTC
    // ---------------------------------------------------------

    // WGM12 = 1
    // Le Timer1 fonctionne en mode CTC
    // (Clear Timer on Compare Match).
    //
    // Le compteur est remis à zéro lorsqu'il atteint OCR1A.
    TCCR1B |= (1 << WGM12);


    // ---------------------------------------------------------
    // Sélection du prédiviseur
    // ---------------------------------------------------------

    // CS12 = 1 et CS10 = 1
    // CS11 = 0
    //
    // Cela sélectionne un prédiviseur de 1024.
    //
    // Horloge du Timer1 :
    // 16 MHz / 1024 = 15625 Hz
    TCCR1B |= (1 << CS12) | (1 << CS10);


    // ---------------------------------------------------------
    // Autorisation de l'interruption de comparaison A
    // ---------------------------------------------------------

    // OCIE1A = 1
    //
    // Autorise l'interruption lorsque le Timer1 atteint
    // la valeur contenue dans OCR1A.
    TIMSK1 |= (1 << OCIE1A);


    // Réactive les interruptions globales.
    interrupts();
}


// ============================================================
// ROUTINE D'INTERRUPTION
// ============================================================

// Cette fonction est appelée automatiquement chaque fois
// que le Timer1 atteint la valeur OCR1A.
ISR(TIMER1_COMPA_vect) {
```

```
  // Incrémente le compteur à chaque interruption.
  compteur++;
}

void loop() {

  // Affiche la valeur actuelle du compteur sur le moniteur série.
  Serial.println(compteur);


  // Attente de 1 seconde.
  // Le programme principal attend pendant cette durée,
  // mais les interruptions continuent à fonctionner.
  delay(1000);
}
```